



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 1 Laboratuvar Föyü

C'den C++'a Geçiş

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>🕒 **Tahmini süre:** 2 saat💻 **Gereksinimler:** C++20 derleyici (g++/clang++), CMake, bir metin editörü

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim 📣 Neden Önemli?

Bir C++ projesi büyüdüğünde tek bir dosya artık yetmez: kodu birden çok .cpp'ye böler, sonra hepsini birlikte derlersiniz. Bu noktada, bir dosya tek başına sorunsuz derlendiği hâlde en sonda “`undefined reference`” hatasıyla karşılaşmak sık yaşanan bir sıkıntıdır — derlemenin ve bağlamanın iki ayrı aşama olduğunu elinizle görmeden bu hata anlamsız kalır. Bu lab'da ilk C++ programınızı yazacak, referans/`auto`/`const` gibi C++'ın yeni “sözlerini” deneyecek ve kendi `geometri::Nokta` bileşeninizi header/source ayrımıyla sıfırdan kurup CMake ile derleyerek bu aşamaları somut olarak yaşayacaksınız.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu föy YG I ders notu Bölüm 1 (C'den C++'a Geçiş) üzerine kuruludur ve şu ön bilgiyi hazır varsayar: C'den — `gcc` ile tek dosya derleme, `#include`, `printf/scanf`, `struct`, işaretçi/adres kavramı ve fonksiyon tanımı/çağırısı; Python'dan — değişkenin tip belirtmeden kullanılabilirdiği, yorumlayıcının satır satır çalıştırdığı bir dil deneyimi (C++'ın derlenen/statik tipli yapıyla karşıtlık kurmak için hatırlanır). Bu hafta o bilgiyi C++'a taşırsınız; yeni bir dili sıfırdan öğrenmezsiniz.

Bu föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (kodu yazın, ardından verilen kabuk komutunu çalıştırın), sonra kendi sonucunuzu *Beklenen* kutusu ve *Çıktı* ile karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Bir şey tutmazsa, sonraki adıma geçmeden o adımı düzeltin — her adım bir öncekinin üstüne inşa edilir. Her 3–4 adımda bir gelen *Kontrol Noktası*, o ana kadar yaptıklarınızı bir arada sınamanıza yarar. Adımlar ders notu Bölüm 1'in dört ünitesiyle sırayla ilerler; her bölümün başında ilgili üniteye bir atıf bulacak, takıldığınızda oraya dönebileceksiniz.

C ile işaretçi, `struct` ve fonksiyon yazmayı, Python ile de tip belirtmeden hızlıca kod yazmayı zaten biliyorsunuz. Bu föyde aynı düşünme biçimini C++'a taşıyacaksınız: C++ de C gibi derlenip bağlanan bir dildir, ama daha sıkı tip kontrolü, `namespace` ile isim düzeni, referanslar ve tip çıkarımı gibi yeni “sözler” tutar. Sırasıyla ilk programınızı yazacak, `<iostream>` ile girdi/çıkı alacak, referansları deneyecek, `auto/const` kullanacak ve son olarak kendi küçük `geometri::Nokta` bileşeninizi header/source ayrımıyla kurup CMake ile derleyeceksiniz.

1 Neden C++? İlk Programınız

(bkz. Ders notu Bölüm 1, Ünite 1 — Neden C++? C/Python'dan Köprü; derleme + bağlama modeli.)

Adım 1: İlk C++ programınızı yazın

? Ne yapacağız? merhaba.cpp adlı bir dosya oluşturacak, içine `std::cout` kullanan minimal bir C++ programı yazacak, ardından `g++` ile derleyip çalıştıracaksınız.

i Neden? C++'ta da derleme + bağlama modeli C'deki ile aynıdır: kaynak dosya önışlem-ciden geçer, derlenir, bağlanır. Aradaki fark çalışma modelinde değil, dilin tuttuğu sözlerdedir — C++'ın `std::cout` akışı, C'nin `printf`'inden farklı olarak yazdığınız değerin tipini derleme zamanında bilir. Bu ilk adımda önce C'deki tanıdık hâlini, sonra C++ karşılığını yan yana göreceksiniz.

Önce hatırlatma amaçlı, C'deki tanıdık hâli:

```
5 int main(void) {
6     printf("Merhaba, C++!\n");
7     return 0;
8 }
```

Şimdi aynı programın C++ karşılığı — merhaba.cpp dosyasına bunu yazın:

```
5 int main() {
6     std::cout << "Merhaba, C++!\n";
7     return 0;
8 }
```

İki satır farklıdır: `#include <stdio.h>` yerine `#include <iostream>` gelir, `printf("...")` yerine `std::cout << "..."` kullanılır. `std::cout`, "standart karakter akışı" anlamına gelen `std` ad alanındaki `cout` nesnesidir; `<<` operatörü verinin akışa doğru "gönderildiğini" gösterir — C'nin format string'i (`%d`, `%s`) burada yoktur, çünkü `<<` her tipi kendi bilerek yazdırır.

>_ Terminal

```
$ g++ -std=c++20 merhaba.cpp -o merhaba
$ ./merhaba
```

👁 Beklenen sonuç: Derleme hatasız tamamlanır; çalıştırdığınızda ekranda tek satırlık bir selam görürsünüz.

🖥 Beklenen Çıktı

```
Merhaba, C++!
```

Ortak Hata / Tuzak

`-std=c++20` bayrağını unutmak yaygın bir hatadır: derleyici o zaman eski bir C++ standardına düşer ve bu föyde ileride kullanacağınız bazı sözdizimleri (aralık tabanlı `for`, bazı `auto` kullanımları) beklenmedik hatalar verebilir. Bu föydeki her `g++` komutunda bayrağı daima yazın.

2 Giriş/Çıkış, `std::string` ve Referanslar

(bkz. Ders notu Bölüm 1, Ünite 2 — `namespace`, `<iostream>`, `std::string`, Referanslar.)

Adım 2: <iostream> ile kullanıcıdan veri okuyun

? Ne yapacağız? `giris_ornekleri.cpp` dosyasında, `std::cin` ile kullanıcıdan bir isim okuyup `std::cout` ile selamlayacak, ardından iki `std::string` değişkenini birleştirip uzunluğunu yazdıracaksınız.

i Neden? `std::cin` ve `std::cout`, C'nin `scanf/printf` çiftinin C++ karşılığıdır; ama `>>/<<` operatörleri hedef değişkenin tipini derleme zamanında bildiğinden, `scanf("%d", ...)` yazıp yanlışlıkla bir `int` yerine `%s` kullanma türünden format string hataları C++'ta ortadan kalkar. `std::string` da C'nin `char*/karakter dizisine` kıyasla uzunluk ve birleştirme gibi işlemleri doğrudan bir üye fonksiyon veya operatörle sunar; elle `strlen/strcat` çağırmanız gerekmez.

```
19 std::string isim;
20 std::cout << "Adınız nedir? ";
21 std::cin >> isim;
22 std::cout << "Merhaba, " << isim << "!\n";
```

`std::cin >> isim;` ifadesi, kullanıcı klavyeden bir kelime yazıp Enter'a basana kadar bekler ve okuduğu metni `isim` değişkenine yazar. Aynı dosyada, `std::string` ile kısa bir birleştirme örneği de var:

```
26 std::string ad = "Ada";
27 std::string soyad = "Lovelace";
28 std::string tamAd = ad + " " + soyad;
29 std::cout << tamAd << " (" << tamAd.size() << " karakter)\n";
```

`+` operatörü burada iki string'i uçtan uca ekler; `.size()` üye fonksiyonu ise sonucun kaç karakter uzunluğunda olduğunu döndürür — C'de bu işlem için `strcat` ve `strlen` çağırmanız, üstelik hedef dizinin yeterince büyük olduğundan siz emin olmanız gerekirdi.

>_ Terminal

```
$ g++ -std=c++20 giris_ornekleri.cpp -o giris_ornekleri
$ ./giris_ornekleri
```

👁 Beklenen sonuç: Program önce adınızı sorar; yazıp Enter'a bastığınızda sizi adınızla selamlar, ardından birleştirilmiş bir ad-soyad ve karakter sayısını yazdırır (bir sonraki adımda ekleyeceğimiz referans satırı henüz çalışmadığı için şimdilik yalnız ilk iki çıktı satırını göreceksiniz).

🖥 Beklenen Çıktı

```
Adınız nedir? Ayse
Merhaba, Ayse!
Ada Lovelace (12 karakter)
```

Adım 3: Referans (int&) ile bir sayıyı ikiye katlayın

? Ne yapacağız? Aynı dosyaya, `int&` parametresi alan bir `ikiyeKatla` fonksiyonu ekleyecek ve bu fonksiyonu `main` içinden çağıracaksınız.

i Neden? C'de bir fonksiyonun çağırının değişkenini değiştirebilmesi için işaretçi (`int*`) geçmeniz, fonksiyon içinde `*` ile "oraya gidip" değeri okuyup yazmanız gerekirdi. C++'ta referans (`int&`), var olan bir değişkene ikinci bir isim (takma ad) verir: çağırın tarafta & yazmanıza, fonksiyon içinde `*` ile dereference etmenize gerek kalmaz — sayı ismi doğrudan çağırının değişkenine işaret eder.

Adım 3 (devamı)

```

6 void ikiyeKatla(int& sayi) {
7     sayi = sayi * 2;
8 }
9
10 void referansOrnegiCalistir() {
11     int sayi = 21;
12     ikiyeKatla(sayi);
13     std::cout << "İkiye katlanan sayi: " << sayi << '\n';
14 }

```

Analoji

Referans, bir kişinin lakabı gibidir: lakap her zaman o kişinin kendisidir, boş olamaz ve sonradan başka bir kişiye "yeniden takılamaz". C'nin işaretçisi ise bir kağıda yazılmış ev adresi gibidir — adres boş (NULL) olabilir, başka bir eve yeniden yönlendirilebilir, adresi okumak için "oraya gitmeniz" (*ptr ile dereference etmeniz) gerekir. `ikiyeKatla(int& sayi)` fonksiyonunda `sayi`, çağırının değişkeninin ikinci ismidir; `sayi = sayi * 2;` satırı doğrudan çağırının değişkenini değiştirir — tıpkı C'de `*ptr = *ptr * 2;` yazmanın yaptığı gibi, ama `*/&` sözdizimi olmadan.

main fonksiyonunuzun sonuna, `return 0;` satırından önce `referansOrnegiCalistir();` çağırısını ekleyin, sonra yeniden derleyip çalıştırın.

>_ Terminal

```

$ g++ -std=c++20 giris_ornekleri.cpp -o giris_ornekleri
$ ./giris_ornekleri

```

Beklenen sonuç: Önceki iki satırın altında artık üçüncü bir satır da görünür: `ikiyeKatla` çağırısından sonra `sayi`'nin 21'den 42'ye çıktığını gösteren satır.

Beklenen Çıktı

```

Adiniz nedir? Ayse
Merhaba, Ayse!
Ada Lovelace (12 karakter)
İkiye katlanan sayi: 42

```

Ortak Hata / Tuzak

Parametre bildiriminde `&` işaretini unutup `void ikiyeKatla(int sayi)` yazmak, fonksiyonu değere göre (*by value*) çağırır: fonksiyon içindeki `sayi`, çağırınınkinden bağımsız bir kopyadır, çarpma yalnız bu kopyayı değiştirir ve çağırındaki değişken hiç değişmeden kalır. Derleyici bu hatayı size bildirmez — çıktıyı görüp değişmemiş bir değerle karşılaşırsınız.

✓ Kontrol Noktası — devam etmeden doğrulayın

`giris_ornekleri.cpp` dosyanız artık isim okuyup selamlıyor, birleştirilmiş bir string ve uzunluğunu yazdırıyor, ve referans parametrelili `ikiyeKatla` ile bir sayıyı ikiye katlıyor. Devam etmeden önce dört çıktı satırının da (isim sorma hariç) doğru sırada görüldüğünü doğrulayın.

3 Tip Çıkarımı (auto) ve Sabitlik (const)

(bkz. Ders notu Bölüm 1, Ünite 3 — auto ve Tip Çıkarımı; const; Modern C++ Tadımı.)

Adım 4: auto ile toplam alıp const referans parametresi kullanın

❓ Ne yapacağız? `auto_const_ornekleri.cpp` dosyasında, `auto` ile tipini derleyiciye bıraktığınız iki değişkenin toplamını alacak, ardından `const std::string&` parametrelili bir `mesajYazdir` fonksiyonu yazıp çağıracağız.

❗ Neden? `auto`, derleyicinin bir ifadenin sağ tarafından değişkenin tipini kendisinin çıkarmasını sağlar — siz hâlâ hangi değeri hesaplayacağınıza karar verirsiniz, yalnız tipin adını elle yazmazsınız. `const std::string&` parametresi ise iki faydayı bir arada verir: `std::string`'i değere göre (kopyalayarak) almak yerine referansla aldığınız için gereksiz bir kopyalama maliyeti oluşmaz, `const` sayesinde de fonksiyonun parametreyi değiştirmeyeceğinden emin olursunuz.

```
13 auto a = 7;
14 auto b = 35;
15 auto toplam = a + b;
16 std::cout << "Toplam: " << toplam << '\n';
```

```
6 void mesajYazdir(const std::string& mesaj) {
7     std::cout << mesaj << '\n';
8 }
```

`auto toplam = a + b;` satırında `toplam`'ın tipi `a/b`'nin tipinden (burada `int`) çıkarılır; tipi elle `int toplam = a + b;` yazsaydınız sonuç aynı olurdu, ama ileride uzun/karmaşık tip adlarıyla (ör. yineleyici tipleri) `auto` okunabilirliği belirgin biçimde artırır. `mesajYazdir` fonksiyonu ise parametresini yalnız okur, hiç değiştirmmez — `const` bu sözü derleyiciye verir ve siz yanlışlıkla `mesaj`'ı değiştirmeye çalışırsanız derleme hatası alırsınız.

Ortak Hata / Tuzak

`auto` her yerde "tipi düşünmeme gerek yok" anlamına gelmez. Tipin okunabilirliği gizlediği yerlerde (ör. bir fonksiyonun döndürdüğü tipin ne olduğu açık değilse) elle tip yazmak daha nettir; `auto` en çok, tipin zaten sağ taraftan apaçık olduğu veya tip adının uzun/ karmaşık olduğu yerlerde kullanışlıdır.

Adım 5: Aralık tabanlı for ile bir diziyi gezin

❓ Ne yapacağız? Aynı dosyada, bir `int` dizisini klasik indeksli `for` yerine aralık tabanlı (*range-based*) `for` ile gezip her elemanı ayrı satırda yazdıracaksınız.

❗ Neden? C'deki klasik `for (int i = 0; i < n; i++)` deseninde hem sınır kontrolünü (`i < n`) hem indeksleme (`dizi[i]`) hata payını siz taşırsınız — sınırı bir eksik/fazla yazmak (*off-by-one*) yaygın bir hatadır. Aralık tabanlı `for`, dizinin/koleksiyonun elemanlarını indeks yazmadan tek tek gezer; sınır hatası yapma ihtimali baştan ortadan kalkar.

```
22 int sayilar[] = {10, 20, 30, 40};
23 for (int sayi : sayilar) {
24     std::cout << sayi << '\n';
25 }
```

`for (int sayi : sayilar)` ifadesi "`sayilar` içindeki her eleman için, o elemanı `sayi` adıyla al" diye okunur; döngü gövdesinde `sayilar[i]` yazmanıza gerek kalmaz.

Adım 5 (devamı)**>_ Terminal**

```
$ g++ -std=c++20 auto_const_ornekleri.cpp -o auto_const_ornekleri
$ ./auto_const_ornekleri
```

🕒 Beklenen sonuç: Önce toplamı, sonra sabit mesajı, sonra dizideki dört sayıyı alt alta gösteren beş satırlık bir çıktı görürsünüz.

🖥 Beklenen Çıktı

```
Toplam: 42
Bu mesaj degistirilemez.
10
20
30
40
```

4 Kütüphanenizi Kurun: Header/Source, namespace, CMake

(bkz. Ders notu Bölüm 1, Ünite 4 — Kütüphaneyi Kurmak: Header/Source, namespace, CMake.)

Adım 6: geometri::Nokta yapısını bildirin (nokta.hpp)

❓ Ne yapacağız? nokta.hpp dosyasında #pragma once ile başlayan, geometri ad alanı için de iki alanlı (x, y) bir Nokta yapısı ve iki serbest fonksiyon bildirimi (topla, yazdir) yazacaksınız.

📌 Neden? Bu adım, Ünite 1'de gördüğünüz derleme + bağlama modelinin somut hâlidir: .hpp yalnız *bildirim* taşır (bu çeviri birimini kullanan her .cpp dosyasına önışlemci tarafından kopyalanır), gerçek *tanım* ise yalnız bir kez nokta.cpp içinde derlenir. #pragma once, aynı header'ın bir çeviri birimine yanlışlıkla iki kez kopyalanmasını (ve buradan doğacak "yeniden tanımlama" hatasını) baştan engeller; namespace geometri ise ileride kütüphanenize başka bileşenler eklediğinizde aynı isimli başka bir Nokta ile çakışmayı önler.

```
7 struct Nokta {
8     double x;
9     double y;
10 };

14 Nokta topla(const Nokta& a, const Nokta& b);
15 void yazdir(const Nokta& n);
```

Bu yapı henüz bir sınıf değildir — yalnız iki genel (*public*) alanı bir arada tutan bir struct'tır (C'deki struct'ın doğrudan devamı). Kapsülleme ve üye fonksiyonlarla gerçek bir sınıfa dönüşümü Hafta 3'te Kitap sınıfıyla göreceksiniz; bu hafta yalnız header/source ayrımı ve namespace deseni kurulur.

Ortak Hata / Tuzak

#pragma once satırını unutmak, dosyanın kendisinde hemen bir hataya yol açmaz — hata, bu header'ı birden fazla .cpp dosyasının (dolaylı olarak) include ettiği bir projede, bağlama aşamasında "yeniden tanımlama" biçiminde ortaya çıkar. Bu yüzden alışkanlık olarak her .hpp dosyasının ilk satırına yazın.

Adım 6 (devamı)

🔗 **Beklenen sonuç:** `nokta.hpp` dosyanız `#pragma once`, namespace `geometri` içinde `Nokta` yapısı ve iki fonksiyon bildirimi içerir; dosya henüz tek başına derlenip çalıştırılabilir bir program oluşturmaz, yalnız bir bildirim dosyasıdır.

Adım 7: Fonksiyon gövdelerini tanımlayın (`nokta.cpp`)

❓ **Ne yapacağız?** `nokta.cpp` dosyasında `nokta.hpp`'yi include edecek, ardından `topla` ve `yazdir` fonksiyonlarının gövdelerini aynı `geometri` ad alanı içinde tanımlayacaksınız.

📌 **Neden?** Menü (header) ile mutfak (source) arasındaki farkı burada somut olarak görürsünüz: `nokta.hpp` müşteriye (bu fonksiyonları çağırarak her `.cpp` dosyasına) yalnız *nelerin* mevcut olduğunu söyler; `nokta.cpp` bu işlemlerin gerçekte *nasil* yapıldığını barındırır. İki parametrenin de `const Nokta&` olması, hem gereksiz kopyalamadan kaçınmanızı hem de fonksiyonların girdilerini değiştirmeyeceğinden emin olmanızı sağlar — Ünite 3'te gördüğünüz `const` referans parametresinin aynı deseni.

```
8 | Nokta topla(const Nokta& a, const Nokta& b) {
9 |     return Nokta{a.x + b.x, a.y + b.y};
10| }

14| void yazdir(const Nokta& n) {
15|     std::cout << "(" << n.x << ", " << n.y << ")";
16| }
```

`topla`, iki noktanın `x` ve `y` alanlarını ayrı ayrı toplayıp yeni bir `Nokta` döndürür; `yazdir` ise bir noktayı (`x`, `y`) biçiminde, satır sonu koymadan ekrana yazar — satır sonunu bir sonraki adımda `nokta_demo.cpp` içinde siz eklersiniz.

🔗 **Beklenen sonuç:** `nokta.cpp` dosyanız artık `nokta.hpp`'de bildirilen iki fonksiyonun da tam tanımını içerir; dosyayı tek başına `g++ -std=c++20 -c nokta.cpp -o nokta.o` ile derlediğinizde hatasız bir nesne dosyası oluşur (henüz `main` içermediği için tek başına çalıştırılabilir değildir).

✓ Kontrol Noktası — devam etmeden doğrulayın

`nokta.hpp` dosyanızda `#pragma once`, `Nokta` yapısı ve iki bildirim var; `nokta.cpp` dosyanızda bu iki fonksiyonun tam tanımı var. Devam etmeden önce her iki dosyada da namespace `geometri { ... }` bloğunun açılıp kapandığından emin olun.

Adım 8: CMake ile derleyin ve çalıştırın

❓ **Ne yapacağız?** `nokta_demo.cpp` içinde iki `Nokta` nesnesi oluşturup `auto` ile toplayan bir `main` yazacak, ardından elle `g++` komutu yazmak yerine bir `CMakeLists.txt` dosyasıyla projeyi derleyeceksiniz.

📌 **Neden?** Şu ana kadar her programı tek bir `g++` komutuyla derlediniz; ama bir proje birden fazla `.cpp` dosyasından oluştuğunda (burada `nokta_demo.cpp` + `nokta.cpp`) bu komut uzar ve dosya sayısı arttıkça elle yönetmesi zorlaşır. CMake, derleme + bağlama adımlarını elle komut yazmadan yöneten bir araçtır: hangi `.cpp` dosyalarının hangi çalıştırılabilir dosyada birleşeceğini bir kez tanımlarsınız, CMake gerisini halleder.

```
7 |     geometri::Nokta a{1.0, 2.0};
8 |     geometri::Nokta b{3.0, 4.0};
9 |     auto toplam = geometri::topla(a, b);
```

Adım 8 (devamı)

```

10
11   geometri::yazdir(a);
12   std::cout << " + ";
13   geometri::yazdir(b);
14   std::cout << " = ";
15   geometri::yazdir(toplam);
16   std::cout << '\n';

```

Şimdi CMakeLists.txt dosyasını oluşturun:

```

1 cmake_minimum_required(VERSION 3.20)
2 project(hafta01_temel CXX)
3 set(CMAKE_CXX_STANDARD 20)
4 set(CMAKE_CXX_STANDARD_REQUIRED ON)
5 add_executable(nokta_demo nokta_demo.cpp nokta.cpp)

```

project(...) projenize bir ad verir; set(CMAKE_CXX_STANDARD 20) her dosyanın C++20 ile derleneceğini bir kez, merkezi olarak belirler (artık her komutta -std=c++20 yazmanız gerekmez); add_executable ise nokta_demo adlı çalıştırılabilir dosyanın hangi .cpp dosyalarından derleneceğini tanımlar.

```

>_ Terminal

$ cmake -B build
$ cmake --build build
$ ./build/nokta_demo

```

👁 **Beklenen sonuç:** İlk komut build/ klasörünü oluşturup derleme sistemini hazırlar, ikinci komut asıl derlemeyi yapar; üçüncü komut programı çalıştırır ve iki noktanın toplamını tek satırda gösterir.

```

🖥 Beklenen Çıktı

(1, 2) + (3, 4) = (4, 6)

```

Görevinizi tamamladıktan sonra, aşağıdakilerden **birini** seçip kendiniz ekleyin:

- `nokta.hpp/nokta.cpp` içine, iki noktanın birbirine eşit olup olmadığını döndüren `bool esitMi(const Nokta&, const Nokta&)` fonksiyonu ekleyin (`x` ve `y` alanlarının ikisi de eşitse `true`); `nokta_demo.cpp` içinde çağırıp sonucu yazdırın.
- `CMakeLists.txt`'e `add_executable(giris_ornekleri ../giris_ornekleri.cpp)` benzeri ikinci bir hedef ekleyip, tek `cmake --build build` komutuyla iki farklı çalıştırılabilir dosyanın birlikte derlendiğini gözlemleyin.

💡 İpucu

Bu haftanın en sık karşılaşılan hataları: (1) `-std=c++20` bayrağını unutup elle `g++` çağırarak; (2) referans parametresinde `&` işaretini atlayıp fonksiyonun çağırıcının değişkenini değiştirmesine şaşırarak; (3) `nokta.hpp`'nin başına `#pragma once` eklemeyi unutup; (4) `nokta.cpp`'yi derleme komutuna (ya da `CMakeLists.txt`'teki `add_executable` satırına) eklemeyi unutup "tanımsız referans" hatası almak; (5) `CMakeLists.txt` değiştirdikten sonra `cmake -B build` adını yeniden çalıştırmayı unutup eski ayarların kullanılmaya devam ettiğini fark etmemek.

✂ Sorun Giderme

Belirti	Olası neden	Çözüm
Program derlenmiyor, "'auto' does not name a type" gibi eski standart hataları	-std=c++20 bayrağı unutuldu	Derleme komutuna -std=c++20 ekleyin
ikiyeKatla çağrıldıktan sonra sayı değişmiyor redefinition of 'struct geometri::Nokta' undefined reference to 'geometri::topla(...)'	Parametre int olarak (değere göre) alındı, & unutuldu nokta.hpp başında #pragma once eksik	ikiyeKatla(int& sayi) biçiminde referans parametresi kullanın Dosyanın en başına #pragma once ekleyin
cmake --build build eski hâliyle çalışıyor gibi görünüyor	nokta.cpp derleme komutuna/CMake hedefine eklenmedi CMakeLists.txt değiştirildi ama cmake -B build yeniden çalıştırılmadı	g++ ... nokta_demo.cpp nokta.cpp -o nokta_demo biçiminde iki dosyayı birlikte verin ya da CMakeLists.txt'teki add_executable satırını kontrol edin cmake -B build komutunu yeniden çalıştırıp ardından cmake --build build deyin

5 • İzleme ➡ Aktarım — Başka Nerede?

Bugün kurduğunuz iskelet tek bir örneğe özgü değildir; dönem boyunca aynı kalıp tekrar tekrar karşınıza çıkar:

- **Kitaplık projesine:** Bu haftanın `nokta.hpp/nokta.cpp` + `namespace` + CMake deseni, Hafta 3'te ilk gerçek sınıfınız olan `kitaplik::Kitap` için birebir tekrar edilir: `kitap.hpp` bildirimi, `kitap.cpp` tanımı taşır, ikisi bir `kitaplik` ad uzayında birleşir. Bugün elle yaşadığınız derleme + bağlama modeli orada da aynen işler — iki `.cpp`'yi hedefe eklemeyi unutursanız yine “`undefined reference`” alırsınız.
- **Başka bileşenlere:** `const` referansla parametre geçme (`const Nokta&`), büyük nesneleri kopyalamadan ve değiştirmeden fonksiyonlara vermenin standart yoludur; ileride `Kitap`, `Vektor2B` ya da `Materyal` gibi her tipte bu alışkanlığı sürdürürsünüz. Referans (`int&`) ile bir değişkeni yerinde değiştirme deseni de aynı biçimde taşınır.
- **İleriye:** Bu hafta gördüğünüz referans ve RAIİ'ye ilk bakış, Hafta 2'de C++'ın bellek modeli (stack/heap, `new/delete`) işlenirken doğrudan zemin oluşturur; `.hpp/.cpp` + CMake iskeleti ise sonraki her haftada aynen yeniden kullanılır.